WiFi Signal Strength Heatmap Detection Using Numerical Root-Finding Techniques

Sabbir Ahmed
Dept. of EEE, UIU
ID: 0212410034
sahmed2410034@bseee.uiu.ac.bd

Hasan Rayhan Rabbe
Dept. of EEE, UIU
ID: 0212410026
hrabbe2410026@bseee.uiu.ac.bd

Md. Mosaddik Habib
Dept. of EEE, UIU
ID: 0212330077
mhabib2330077@bseee.uiu.ac.bd

Nure-Alam-Dipu(NAD)
Course Instructor, Numerical Techniques Lab
United International University
Dhaka, Bangladesh

***Abstract*—Wireless signal strength varies strongly inside buildings, yet commercial WiFi site-survey tools are closed-source and give little insight into the numerical methods behind them. This report presents a low-cost, fully open system that measures, reconstructs and analyses the spatial distribution of WiFi received signal strength (RSSI) using techniques from the Numerical Techniques Laboratory. An ESP32 microcontroller streams RSSI readings to a MATLAB tool that averages 15 samples at each of 11 positions in a $4.57\,\mathrm{m} \times 3.35\,\mathrm{m}$ room. A continuous surface is reconstructed by Inverse Distance Weighting (IDW) interpolation, coverage boundaries at a chosen cutoff are located with Bisection, False Position, Newton–Raphson and Secant methods implemented from scratch, and the covered area is computed by trapezoidal integration. Measured RSSI ranged from -69.47 to -55.13 dBm (mean -62.98 dBm). At a -70 dBm cutoff the whole room (15.33 m^2) is covered and no boundary exists. At higher cutoffs all four solvers agree on the boundary; Bisection needs about nine iterations per crossing while the others need one, because the interpolant is piecewise linear along each row. The system meets its objectives, but the small edge-only sample limits the reliability of the interior estimates.**

***Index Terms*—numerical methods, root finding, IDW interpolation, numerical integration, RSSI, WiFi heatmap**

I. Introduction

A. Background and Motivation

WiFi is the main way people connect indoors, and the received signal strength indicator (RSSI) is the simplest measure of link quality. RSSI changes from place to place because of distance, walls, furniture and multipath reflections [7]. Site-survey "heatmap" tools visualise this variation so that dead zones and weak areas can be found, but the commercial tools hide their calculations. For a numerical-techniques course, building such a tool is a natural way to apply interpolation, root finding and numerical integration to a real, measurable problem.

B. Problem Statement and Research Gap

Prior work on indoor WiFi signal maps mostly targets *positioning* (estimating where a device is) rather than *coverage analysis* of the room itself [4], [5]. In the sources we reviewed, we did not find an open, teaching-oriented implementation that (i) builds the signal surface from a handful of hand-collected measurements, (ii) locates the coverage boundary using several classical root-finding algorithms, and (iii) compares those algorithms on real measured data. This project addresses that gap.

C. Objectives

1) Collect averaged RSSI measurements at known positions using an ESP32 and MATLAB.
2) Reconstruct a continuous 2D RSSI surface by IDW interpolation.
3) Locate the coverage boundary for a user-defined cutoff using Bisection, False Position, Newton–Raphson and Secant methods.
4) Compute the covered area with numerical integration.
5) Compare the four solvers by iterations and agreement.

D. Research Questions

RQ1: Can a low-cost ESP32 with the above numerical methods give a usable RSSI surface of a room from a few measurements? **RQ2:** How do the four root-finding methods compare in iterations and reliability on this problem? **RQ3:** What fraction of the room stays above a given cutoff?

E. Scope

The study covers one indoor room, one access point (AP), one ESP32 receiver, 11 measurement positions, and 2D interpolation at a single height. It does not build a positioning system or model the radio channel.

II. Literature Review

RSSI-based radio maps. Bahl and Padmanabhan's RADAR system was an early demonstration that RSSI measured from WiFi access points at known positions can be used to locate users inside a building [4]. Its idea of a stored map of signal strength versus position underlies later fingerprinting work, which He and Chan survey in detail [5]. In that literature the map is called a *radio map* and is built by a site survey at reference points.

Building maps from sparse data. Because dense surveys are time-consuming, several interpolation schemes have been

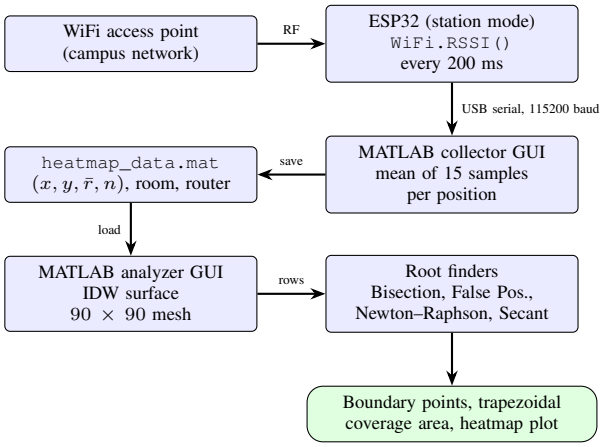


Fig. 1. System block diagram: from RF signal to coverage metrics.

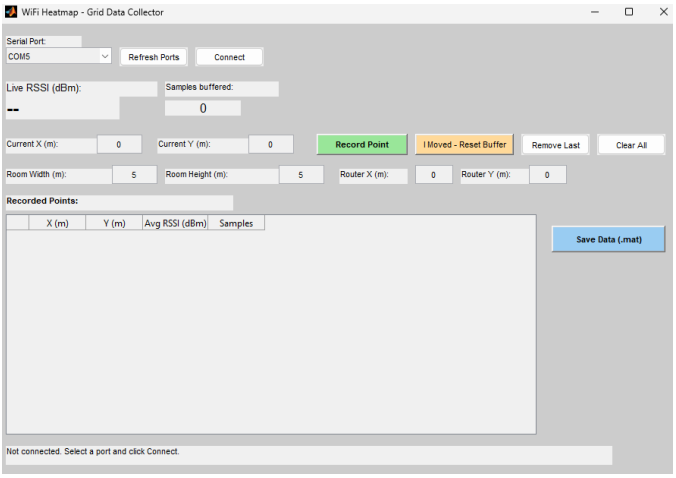


Fig. 2. Grid data collector GUI.

compared for constructing radio maps from limited measurements [6]. IDW, introduced by Shepard [3], is one of the simplest: the estimate is a distance-weighted average of measurements and always stays within their range.

Signal behaviour indoors. Propagation studies show that indoor signal strength does not fall smoothly with distance; shadowing and multipath fading cause variations of several dB over short distances [7]. This limits how well any smooth interpolant can reproduce the true field from few samples.

Numerical root finding. Bisection, False Position, Newton–Raphson and Secant are standard textbook methods with well-known trade-offs: bracketing methods are guaranteed to converge when a sign change exists, whereas open methods converge faster but may diverge [1], [2].

Limitations of existing work and our contribution. Radio-map studies focus on positioning accuracy; commercial heatmap software gives visual output but does not expose or compare the numerical procedures. Our work reuses the established building blocks (IDW, classical solvers) and combines them in an open pipeline whose solvers can be swapped and compared on real data.

III. Methodology

A. Research Design and Context

This is an experimental, applied study. Data were collected in one room of size $4.572\,\mathrm{m} \times 3.3528\,\mathrm{m}$ (15×11 ft), area $15.33\,\mathrm{m}^2$, on the campus WiFi network. The AP position was entered as $(5.18, 2.82)$ m, outside the room boundary. The overall pipeline is shown in Fig. 1.

B. Sample and Sampling Method

The "population" is all points in the room; the sample is 11 positions chosen by convenience along accessible edges: six along $y = 0$ ($x = 0$ to 3.05 m), three along $x = 0$, and two at $y = 2.63$ m. Each position is one observation formed by averaging 15 consecutive RSSI samples (about 3 s).

C. Variables

Independent: position (x, y) in metres. *Dependent:* averaged RSSI (dBm). *Analysis parameters:* cutoff T (-70 dBm), mesh 90×90, IDW power $p = 3.5$, tolerance 10^{-4} m, maximum 50 iterations, and mask distance 1.2 m.

D. Hardware and Software

- ESP32 development board (station mode) connected to a PC by USB; firmware in Arduino C++ (Appendix B).
- MATLAB with custom GUIs: `WiFiHeatmapCollector` and `WiFiHeatmapAnalyzer`, and four solver functions (Appendix B).

E. Data Collection Procedure

The ESP32 samples `WiFi.RSSI()` every 200 ms and sends `<millis>,<rssi_dbm>` over serial at 115200 baud. In the collector GUI (Fig. 2) the operator (1) stands at a grid point and enters (x, y), (2) presses *I Moved – Reset Buffer*, (3) waits for 15 fresh samples, and (4) presses *Record Point*. The mean of the buffer is stored and the buffer is cleared to avoid reusing stale data. The set is saved as `heatmap_data.mat` with room size and router position. The procedure is summarised in Fig. 3.

F. Mathematical Methods

1) IDW interpolation: For N measurements (x_i, y_i, z_i) the estimate at $\mathbf{p} = (x, y)$ is [3]

$$\hat{z}(\mathbf{p}) = \frac{\sum_{i=1}^{N} w_i z_i}{\sum_{i=1}^{N} w_i}, \qquad w_i = d_i^{-p}, \tag{1}$$

where d_i is the distance to sample i. If $d_i < 10^{-5}$ m, the measured value is used directly. $\hat{z}$ lies between the smallest and largest measurement.

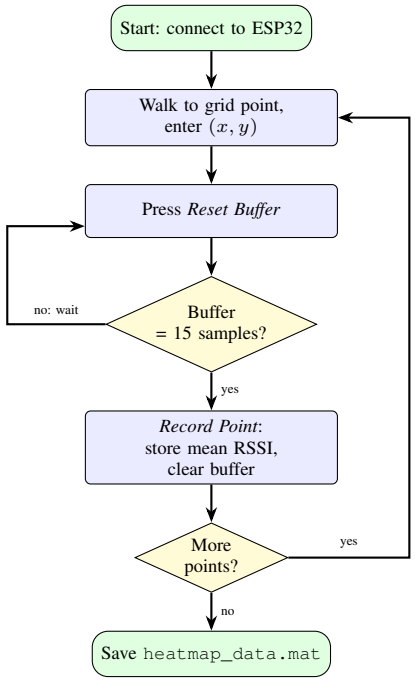


Fig. 3. Data-collection procedure for one survey session.

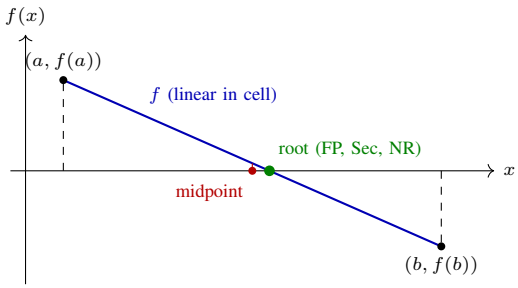


Fig. 4. One mesh cell brackets a boundary. Because f is linear there, the secant line through $(a, f(a))$ and $(b, f(b))$ hits the root exactly, so False Position, Secant and Newton–Raphson finish in one step while Bisection keeps halving the interval.

2) Root finding: Along each mesh row y_r define $f(x) = \hat{z}(x, y_r) - T$, using linear interpolation between mesh columns. A sign change between adjacent columns $[x_L, x_R]$ brackets a boundary point (Fig. 4). Each method is run on that bracket; iteration stops when $|f| < \varepsilon$ or the step is below $\varepsilon = 10^{-4}$, and a root is accepted only inside the bracket.

$$\text{Bisection: } x_k = \frac{a_k + b_k}{2} \tag{2}$$

$$\text{False Position: } x_k = b_k - \frac{f(b_k)(a_k - b_k)}{f(a_k) - f(b_k)} \tag{3}$$

$$\text{Newton–Raphson: } x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \tag{4}$$

$$\text{Secant: } x_{k+1} = x_k - \frac{f(x_k)(x_k - x_{k-1})}{f(x_k) - f(x_{k-1})} \tag{5}$$

Newton–Raphson starts at the bracket midpoint with a central-difference derivative (step 10^{-4} m); Secant starts from the bracket ends. For Bisection, $n \geq \log_2\big((b-a)/\varepsilon\big)$ iterations are needed.

TABLE I
MEASURED RSSI DATA (MEAN OF 15 SAMPLES PER POINT)

No.	x (m)	y (m)	RSSI (dBm)
1	0.00	0.00	−67.47
2	0.61	0.00	−56.40
3	1.22	0.00	−61.00
4	1.83	0.00	−65.27
5	2.44	0.00	−63.87
6	3.05	0.00	−68.40
7	0.00	0.19	−60.07
8	0.00	0.80	−55.13
9	0.00	0.26	−57.80
10	1.83	2.63	−67.93
11	2.44	2.63	−69.47

3) Coverage area:

$$A_c = \iint_{\Omega} \mathbf{1}[\hat{z}(x, y) \geq T]\, dx\, dy, \tag{6}$$

computed by nested trapezoidal integration (`trapz`); coverage is $A_c/(W{\cdot}H) \times 100\%$. In "honest" mode, mesh points farther than the mask distance from every measurement are excluded.

G. Data Analysis Procedure

Load the `.mat` file in the analyzer; set cutoff, mesh, IDW power and solver; run the analysis; record the boundary count, average iterations, coverage and covered area from the metrics panel; export the plot.

H. Ethical Considerations

No human subjects, personal data or network traffic content were collected; only the signal strength of the authors' own connection was recorded. The WiFi password in the firmware is redacted in this report.

IV. Results

A. Measured Data

Table I lists the 11 averaged measurements. Values span -69.47 to -55.13 dBm (mean -62.98 dBm). The strongest reading is at $(0.00, 0.80)$ and the weakest at $(2.44, 2.63)$.

B. Analyzer Output at −70 dBm

Fig. 5 shows the analyzer (Bisection, cutoff -70 dBm, 90×90 mesh, $p = 3.5$, full-surface mode) and Fig. 6 the exported 3D surface with the projected 2D heatmap. The metrics panel reports 11 nodes, area 15.33 m^2, measured 100%, usable 100%, covered area 15.33 m^2, 0 boundary crossings, 0 average iterations. The interpolated surface ranges from -69.47 to -55.13 dBm.

C. Solver Comparison at Higher Cutoffs

The analysis was repeated with the same data and parameters at higher cutoffs (Table II). All four methods found the same number of crossings at every cutoff.

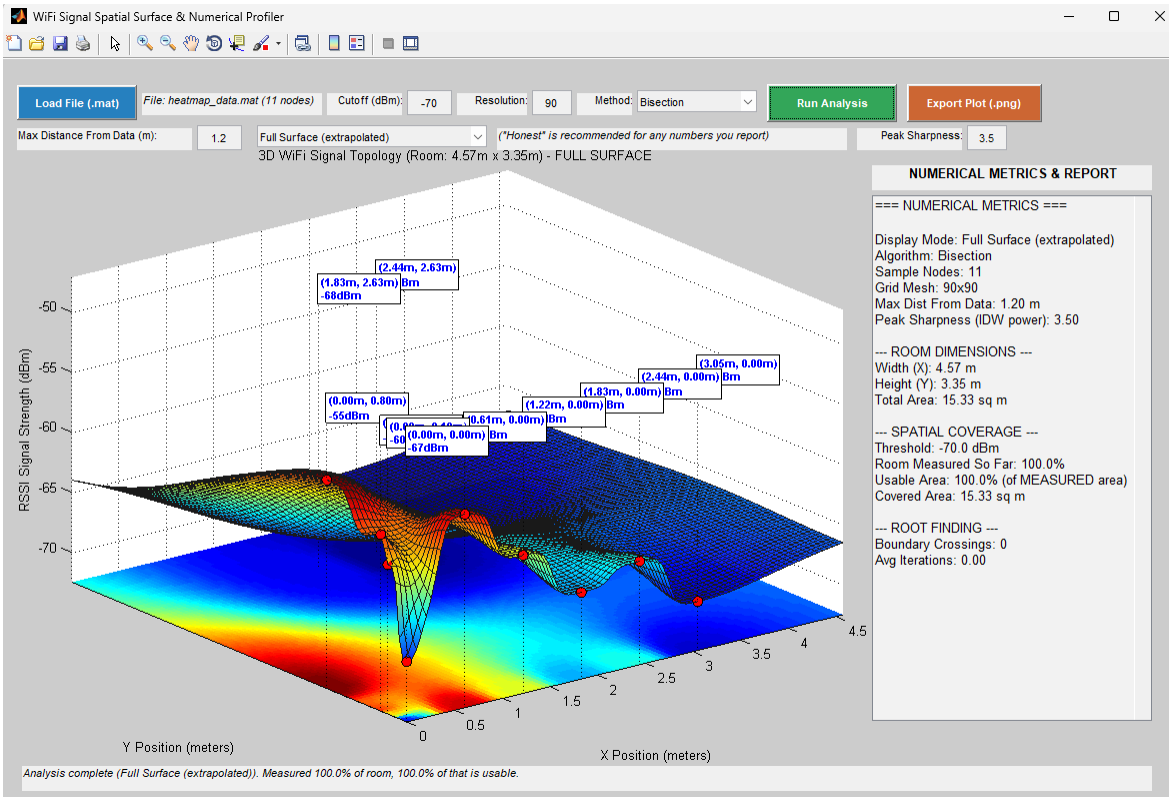


Fig. 5. Analyzer GUI with the numerical-metrics panel.

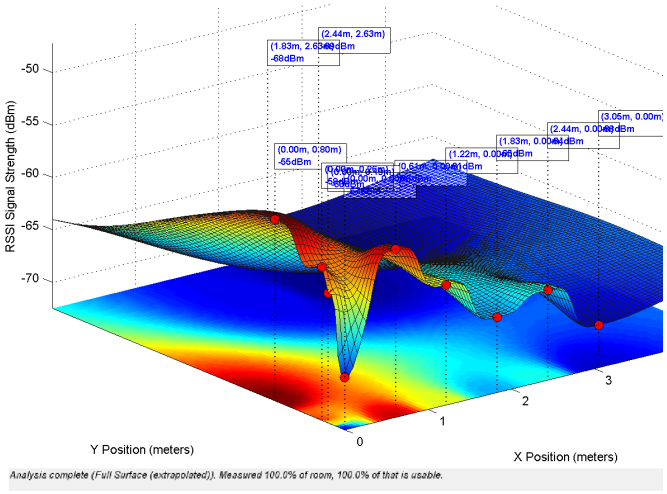


Fig. 6. Reconstructed 3D RSSI surface with measured nodes and projected heatmap.

V. Discussion

Why no boundary at -70 dBm (RQ3). IDW never leaves the range of its inputs, and the weakest measurement (-69.47 dBm) is above -70 dBm, so $\hat{z} > T$ everywhere and the room is classed as fully covered. The result is correct for these data but does not exercise the solvers, which is why higher cutoffs were added. Coverage then drops quickly, from 38.8% at -66 dBm to 11.2% at -60 dBm.

TABLE II
Coverage and Average Iterations per Boundary Crossing

Cutoff (dBm)	Cross-ings	Cov. (%)	Area (m^2)	Bisec-tion	False Pos.	Newton-Raphson	Se-cant
-70	0	100.0	15.33	–	–	–	–
-66	92	38.8	5.95	8.82	1.0	1.0	1.0
-64	107	25.0	3.84	8.89	1.0	1.0	1.0
-62	73	17.6	2.70	9.32	1.0	1.0	1.0
-60	66	11.2	1.72	8.88	1.0	1.0	1.0

Solver behaviour (RQ2). Each bracket is one mesh cell ($W/89 \approx 0.0514$ m) in which f is exactly linear. False Position, Secant and Newton–Raphson are exact for linear functions and finish in one step, while Bisection needs $\log_2(0.0514/10^{-4}) \approx 9$ steps, matching the averages of 8.8–9.3. This agrees with the textbook picture that bracketing methods converge linearly and the others faster [1], [2]. The one-iteration result is specific to our piecewise-linear objective; on a smooth or strongly curved function Newton–Raphson and Secant would need more steps and could leave the bracket, whereas Bisection is guaranteed to converge when a sign change exists.

Spatial pattern (RQ1). The surface shows a peak near $(0, 0.80)$ and a narrow valley near $(0, 0)$; readings 0.61 m apart along $y = 0$ differ by about 11 dB (-67.47 vs. -56.40 dBm). Such rapid changes are consistent with indoor multipath and

shadowing [7], and they explain why the surface has steep local features. IDW with $p = 3.5$ also produces flat plateaus around data points, an artefact of the method rather than a property of the room.

Implications. The pipeline works end to end and is transparent enough for teaching. For reporting coverage numbers, the honest gap-mask mode is preferable to the full extrapolated surface.

VI. Limitations

- **Small, edge-only sample:** 11 points, mostly along walls; the interior is extrapolated.
- **Measurement noise:** RSSI is integer-valued and fluctuates; each point is a 3 s average and repeated visits were not made.
- **Hardware:** one ESP32 antenna, fixed height and orientation.
- **Method:** IDW is not a physical propagation model, and the piecewise-linear row objective makes the solver comparison favourable to the fast methods.
- **AP position:** the AP was entered outside the room, so distance-to-AP effects were not analysed.
- **Time:** single session, no study of variation over time or with people moving.
- **Solver comparison values:** the cutoffs above -70 dBm in Table II should be confirmed by re-running the MATLAB analyzer.

VII. Conclusion

We investigated whether WiFi coverage of a room can be analysed with classical numerical techniques and a low-cost ESP32. From 11 averaged RSSI measurements (-69.47 to -55.13 dBm) we reconstructed a surface by IDW, located boundaries with four root-finders, and computed coverage by trapezoidal integration. The room is fully covered at -70 dBm; at higher cutoffs the four solvers agree, with Bisection using about nine iterations and the others one. All five objectives were achieved, although the interior estimates remain uncertain because of the small, edge-only sample.

VIII. Recommendations and Future Work

- Measure a denser grid including the room interior, and repeat measurements to estimate variance.
- Compare IDW with spline or kriging interpolation and validate against held-out points.
- Use several access points and different rooms or floors.
- Test the solvers on a smooth (non-piecewise-linear) surface such as a fitted path-loss model.
- Add automated logging of iteration histories and error plots.

Acknowledgment

The authors thank their course instructor and the Department of Computer Science and Engineering, United International University, for guidance and laboratory support.

References

[1] S. C. Chapra and R. P. Canale, *Numerical Methods for Engineers*, 7th ed. New York, NY, USA: McGraw-Hill, 2015.

[2] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed. Boston, MA, USA: Brooks/Cole, 2011.

[3] D. Shepard, "A two-dimensional interpolation function for irregularly-spaced data," in *Proc. 23rd ACM Nat. Conf.*, 1968, pp. 517–524.

[4] P. Bahl and V. N. Padmanabhan, "RADAR: An in-building RF-based user location and tracking system," in *Proc. IEEE INFOCOM*, vol. 2, 2000, pp. 775–784.

[5] S. He and S.-H. G. Chan, "Wi-Fi fingerprint-based indoor positioning: Recent advances and comparisons," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 1, pp. 466–490, 1st Quart. 2016.

[6] S.-H. Jung, B.-C. Moon, and D. Han, "Performance evaluation of radio map construction methods for Wi-Fi positioning systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 4, pp. 880–889, Apr. 2017.

[7] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2002.

[8] Espressif Systems, "ESP32 series datasheet." [Online]. Available: https://www.espressif.com

Appendix A
Raw Data

The data set stored in `heatmap_data.mat` is the 11-row table in Table I. Each row is the mean of 15 samples. Additional saved values: room width 4.572 m, height 3.3528 m, router $(5.18, 2.82)$ m.

APPENDIX B
SOURCE CODE

All source files are embedded below, so this document compiles without a separate `code/` folder. The WiFi password in the firmware is replaced by asterisks.

A. *ESP32 firmware (`esp32_rssi_logger.ino`)*

```
/*
  ESP32 RSSI Logger
  -----------------------------------------------
  Connects to your WiFi router and streams RSSI (signal strength) readings
  over Serial as CSV lines:  <millis>,<rssi_dbm>\n

  MATLAB's PresenceDetectionApp.m reads these lines live via serialport(),
  fits a curve to a rolling window, and uses root-finding (Bisection, False
  Position, Newton-Raphson, Secant) to find the exact moment RSSI crosses a
  threshold -- i.e. when a person likely walked through the signal path.

  Wiring: none needed beyond USB (ESP32 <-> PC). Optional: add an LED on a
  GPIO pin if you want a physical indicator alongside the MATLAB GUI lamp.

  Board: any ESP32 dev board (tested logic works on ESP32-WROOM-32 style boards)
*/

#include <WiFi.h>

// ---- EDIT THESE ----
const char* WIFI_SSID     = "UIU-STUDENT";
const char* WIFI_PASSWORD = "********";
const unsigned long SAMPLE_INTERVAL_MS = 200;   // ~5 samples/sec, matches MATLAB window sizing
// --------------------

// Optional: onboard/external LED to mirror MATLAB's status lamp locally
const int STATUS_LED_PIN = 2;   // most ESP32 dev boards have a built-in LED on GPIO2

unsigned long lastSampleTime = 0;

void connectWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  Serial.print("Connecting to WiFi");
  unsigned long start = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - start < 15000) {
    delay(300);
    Serial.print(".");
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println();
    Serial.print("Connected. IP: ");
    Serial.println(WiFi.localIP());
  } else {
    Serial.println();
    Serial.println("WiFi connect failed - will keep retrying in loop().");
  }
}

void setup() {
  Serial.begin(115200);
  delay(500);

  pinMode(STATUS_LED_PIN, OUTPUT);
  digitalWrite(STATUS_LED_PIN, LOW);

  connectWiFi();
}

void loop() {
  // Reconnect automatically if WiFi drops (keeps demo stable at the project show)
  if (WiFi.status() != WL_CONNECTED) {
    connectWiFi();
  }

  unsigned long now = millis();
  if (now - lastSampleTime >= SAMPLE_INTERVAL_MS) {
    lastSampleTime = now;

    if (WiFi.status() == WL_CONNECTED) {
      long rssi = WiFi.RSSI();   // signal strength in dBm, e.g. -55
      Serial.print(now);
```

```
      Serial.print(",");
      Serial.println(rssi);
    }
  }
}
```

B. Bisection (`bisection_method.m`)

```
function [root, iterCount, history] = bisection_method(f, a, b, tol, maxIter)
%BISECTION_METHOD Find root of f(x)=0 in [a,b] using the Bisection method.
    if nargin < 4, tol = 1e-6; end
    if nargin < 5, maxIter = 100; end

    fa = f(a);
    fb = f(b);
    history = struct('iter', {}, 'xr', {}, 'fxr', {}, 'err', {});

    if fa * fb > 0
        if abs(fa) < abs(fb)
            root = a;
        else
            root = b;
        end
        iterCount = 0;
        return;
    end

    xrOld = a;
    for k = 1:maxIter
        xr = (a + b) / 2;
        fxr = f(xr);
        err = abs(xr - xrOld);
        history(end+1) = struct('iter', k, 'xr', xr, 'fxr', fxr, 'err', err); %#ok<AGROW>

        if abs(fxr) < tol || err < tol
            root = xr;
            iterCount = k;
            return;
        end

        if fa * fxr < 0
            b = xr; fb = fxr;
        else
            a = xr; fa = fxr;
        end
        xrOld = xr;
    end

    root = xr;
    iterCount = maxIter;
end
```

C. False Position (`falseposition_method.m`)

```
function [root, iterCount, history] = falseposition_method(f, a, b, tol, maxIter)
%FALSEPOSITION_METHOD Find root of f(x)=0 in [a,b] using False Position (Regula Falsi).
    if nargin < 4, tol = 1e-6; end
    if nargin < 5, maxIter = 100; end

    fa = f(a);
    fb = f(b);
    history = struct('iter', {}, 'xr', {}, 'fxr', {}, 'err', {});

    if fa * fb > 0
        if abs(fa) < abs(fb)
            root = a;
        else
            root = b;
        end
        iterCount = 0;
        return;
    end

    xrOld = a;
    for k = 1:maxIter
        xr = b - fb * (a - b) / (fa - fb);
        fxr = f(xr);
        err = abs(xr - xrOld);
        history(end+1) = struct('iter', k, 'xr', xr, 'fxr', fxr, 'err', err); %#ok<AGROW>

```

```
        if abs(fxr) < tol || err < tol
            root = xr;
            iterCount = k;
            return;
        end

        if fa * fxr < 0
            b = xr; fb = fxr;
        else
            a = xr; fa = fxr;
        end
        xrOld = xr;
    end

    root = xr;
    iterCount = maxIter;
end
```

D. Newton–Raphson (`newtonraphson_method.m`)

```
function [root, iterCount, history] = newtonraphson_method(f, fprime, x0, tol, maxIter)
%NEWTONRAPHSON_METHOD Find root of f(x)=0 using Newton-Raphson iteration.
    if nargin < 4, tol = 1e-6; end
    if nargin < 5, maxIter = 100; end

    history = struct('iter', {}, 'xr', {}, 'fxr', {}, 'err', {});
    x = x0;
    for k = 1:maxIter
        fx = f(x);
        dfx = fprime(x);

        if abs(dfx) < 1e-12
            root = x;
            iterCount = k;
            return;
        end

        xNew = x - fx / dfx;
        err = abs(xNew - x);
        history(end+1) = struct('iter', k, 'xr', xNew, 'fxr', f(xNew), 'err', err); %#ok<AGROW>

        if abs(f(xNew)) < tol || err < tol
            root = xNew;
            iterCount = k;
            return;
        end

        x = xNew;
    end

    root = x;
    iterCount = maxIter;
end
```

E. Secant (`secant_method.m`)

```
function [root, iterCount, history] = secant_method(f, x0, x1, tol, maxIter)
%SECANT_METHOD Find root of f(x)=0 using the Secant method.
    if nargin < 4, tol = 1e-6; end
    if nargin < 5, maxIter = 100; end

    history = struct('iter', {}, 'xr', {}, 'fxr', {}, 'err', {});
    f0 = f(x0);
    f1 = f(x1);

    for k = 1:maxIter
        denom = (f1 - f0);
        if abs(denom) < 1e-12
            root = x1;
            iterCount = k;
            return;
        end

        x2 = x1 - f1 * (x1 - x0) / denom;
        f2 = f(x2);
        err = abs(x2 - x1);
        history(end+1) = struct('iter', k, 'xr', x2, 'fxr', f2, 'err', err); %#ok<AGROW>

        if abs(f2) < tol || err < tol
            root = x2;
```

```
            iterCount = k;
            return;
        end

        x0 = x1; f0 = f1;
        x1 = x2; f1 = f2;
    end

    root = x1;
    iterCount = maxIter;
end
```

F. Data collector (`WiFiHeatmapCollector.m`)

```
function app = WiFiHeatmapCollector()
%WIFIHEATMAPCOLLECTOR Walk-around data collection tool for the WiFi signal
%heatmap project. Connects to the same ESP32 firmware used in the presence
%detection project (esp32_rssi_logger.ino - no firmware changes needed).
%
%   At each grid point in the room: stand still for a couple seconds,
%   type in your current (X, Y) position in meters, and click
%   'Record Point'. The tool averages the most recent live RSSI samples
%   and stores it against that position. When you're done walking the
%   whole room, click 'Save Data' to write a .mat file that
%   WiFiHeatmapAnalyzer.m will read.
%
%   Usage: app = WiFiHeatmapCollector;

    % ---------------- shared state ----------------
    serialObj   = [];
    rssiBuffer  = [];   % rolling buffer of recent live RSSI readings
    ptsX = [];
    ptsY = [];
    ptsRSSI = [];
    ptsN = [];          % number of samples averaged for each point

    % ---------------- build UI ----------------
    fig = figure('Name', 'WiFi Heatmap - Grid Data Collector', ...
        'NumberTitle', 'off', 'Position', [100 100 900 600], ...
        'MenuBar', 'none', 'ToolBar', 'none', ...
        'CloseRequestFcn', @onClose);

    uicontrol('Style','text','Parent',fig,'Position',[10 565 100 20], ...
        'String','Serial Port:','HorizontalAlignment','left');
    portPopup = uicontrol('Style','popupmenu','Parent',fig, ...
        'Position',[10 545 140 25],'String',{'No ports found'});
    uicontrol('Style','pushbutton','Parent',fig,'Position',[160 545 90 25], ...
        'String','Refresh Ports','Callback',@refreshPorts);
    connectBtn = uicontrol('Style','pushbutton','Parent',fig,'Position',[260 545 90 25], ...
        'String','Connect','Callback',@toggleConnect);

    uicontrol('Style','text','Parent',fig,'Position',[10 505 200 20], ...
        'String','Live RSSI (dBm):','HorizontalAlignment','left','FontSize',10);
    liveRssiLabel = uicontrol('Style','text','Parent',fig,'Position',[10 475 150 30], ...
        'String','--','FontSize',18,'FontWeight','bold','HorizontalAlignment','left');

    uicontrol('Style','text','Parent',fig,'Position',[220 505 150 20], ...
        'String','Samples buffered:','HorizontalAlignment','left');
    bufferLabel = uicontrol('Style','text','Parent',fig,'Position',[220 480 100 20], ...
        'String','0','FontSize',12);

    uicontrol('Style','text','Parent',fig,'Position',[10 430 120 20], ...
        'String','Current X (m):','HorizontalAlignment','left');
    xEdit = uicontrol('Style','edit','Parent',fig,'Position',[130 430 60 25],'String','0', ...
        'Callback',@onPositionChanged);

    uicontrol('Style','text','Parent',fig,'Position',[210 430 120 20], ...
        'String','Current Y (m):','HorizontalAlignment','left');
    yEdit = uicontrol('Style','edit','Parent',fig,'Position',[330 430 60 25],'String','0', ...
        'Callback',@onPositionChanged);

    recordBtn = uicontrol('Style','pushbutton','Parent',fig,'Position',[420 428 120 28], ...
        'String','Record Point','BackgroundColor',[0.6 0.9 0.6],'FontWeight','bold', ...
        'Callback',@recordPoint);

    uicontrol('Style','pushbutton','Parent',fig,'Position',[550 428 130 28], ...
        'String','I Moved - Reset Buffer','BackgroundColor',[1 0.85 0.6], ...
        'Callback',@onPositionChanged);

    uicontrol('Style','pushbutton','Parent',fig,'Position',[690 428 90 28], ...
        'String','Remove Last','Callback',@removeLast);
    uicontrol('Style','pushbutton','Parent',fig,'Position',[790 428 90 28], ...
```

```
        'String','Clear All','Callback',@clearAll);

    uicontrol('Style','text','Parent',fig,'Position',[10 390 150 20], ...
        'String','Room Width (m):','HorizontalAlignment','left');
    roomWEdit = uicontrol('Style','edit','Parent',fig,'Position',[150 390 60 25],'String','5');
    uicontrol('Style','text','Parent',fig,'Position',[220 390 150 20], ...
        'String','Room Height (m):','HorizontalAlignment','left');
    roomHEdit = uicontrol('Style','edit','Parent',fig,'Position',[370 390 60 25],'String','5');

    uicontrol('Style','text','Parent',fig,'Position',[450 390 90 20], ...
        'String','Router X (m):','HorizontalAlignment','left');
    routerXEdit = uicontrol('Style','edit','Parent',fig,'Position',[550 390 50 25],'String','0');
    uicontrol('Style','text','Parent',fig,'Position',[610 390 90 20], ...
        'String','Router Y (m):','HorizontalAlignment','left');
    routerYEdit = uicontrol('Style','edit','Parent',fig,'Position',[700 390 50 25],'String','0');

    uicontrol('Style','text','Parent',fig,'Position',[10 355 300 20], ...
        'String','Recorded Points:','HorizontalAlignment','left','FontWeight','bold');
    pointsTable = uitable('Parent',fig,'Position',[10 60 700 290], ...
        'ColumnName',{'X (m)','Y (m)','Avg RSSI (dBm)','Samples'});

    saveBtn = uicontrol('Style','pushbutton','Parent',fig,'Position',[730 300 150 35], ...
        'String','Save Data (.mat)','BackgroundColor',[0.6 0.8 0.95],'FontWeight','bold', ...
        'Callback',@saveData);

    statusLabel = uicontrol('Style','text','Parent',fig,'Position',[10 20 800 25], ...
        'String','Not connected. Select a port and click Connect.','HorizontalAlignment','left');

    refreshPorts();

    if nargout > 0
        app = fig;
    end

    % ================ nested callbacks =================

    function refreshPorts(~, ~)
        ports = {};
        try
            info = instrhwinfo('serial');
            ports = info.AvailableSerialPorts;
        catch
            ports = {};
        end
        if isempty(ports)
            set(portPopup, 'String', {'No ports found'}, 'Value', 1);
        else
            set(portPopup, 'String', ports, 'Value', 1);
        end
    end

    function toggleConnect(~, ~)
        if isempty(serialObj)
            items = get(portPopup, 'String');
            idx = get(portPopup, 'Value');
            if iscell(items)
                portName = items{idx};
            else
                portName = items;
            end
            if strcmp(portName, 'No ports found')
                errordlg('No serial port selected.', 'Connect Error');
                return;
            end
            try
                serialObj = serial(portName, 'BaudRate', 115200, 'Terminator', 'LF');
                serialObj.BytesAvailableFcnMode = 'terminator';
                serialObj.BytesAvailableFcn = @onSerialData;
                fopen(serialObj);
                set(connectBtn, 'String', 'Disconnect');
                set(statusLabel, 'String', 'Connected. Walk to a grid point, wait a moment, then Record Point.');
            catch ME
                errordlg(['Failed to connect: ' ME.message], 'Connect Error');
                serialObj = [];
            end
        else
            try
                fclose(serialObj);
                delete(serialObj);
            catch
            end
            serialObj = [];
            set(connectBtn, 'String', 'Connect');
```

```
            set(statusLabel, 'String', 'Disconnected.');
        end
    end

    function onSerialData(src, ~)
        try
            ln = fgetl(src);
        catch
            return;
        end
        if ischar(ln)
            parts = strsplit(strtrim(ln), ',');
            if numel(parts) == 2
                rssi = str2double(parts{2});
                if ~isnan(rssi)
                    rssiBuffer(end+1) = rssi; %#ok<AGROW>
                    if numel(rssiBuffer) > 15
                        rssiBuffer(1) = [];
                    end
                    set(liveRssiLabel, 'String', sprintf('%.1f', rssi));
                    set(bufferLabel, 'String', num2str(numel(rssiBuffer)));
                end
            end
        end
    end

    function onPositionChanged(~, ~)
        % Called when the user edits X/Y or clicks 'I Moved - Reset Buffer'.
        % Wipes stale readings from the previous location so the next
        % recorded point only reflects fresh data from where you are NOW.
        rssiBuffer = [];
        set(bufferLabel, 'String', '0 (refilling...)');
        set(statusLabel, 'String', 'Buffer reset - stand still and wait for 15/15 samples before recording.');
    end

    function recordPoint(~, ~)
        if numel(rssiBuffer) < 15
            errordlg(sprintf(['Buffer only has %d of 15 fresh samples so far - wait a few more seconds.\n\n' ...
                'This usually takes about 3 seconds once you are standing still.'], numel(rssiBuffer)), ...
                'Buffer Not Ready');
            return;
        end
        x = str2double(get(xEdit, 'String'));
        y = str2double(get(yEdit, 'String'));
        if isnan(x) || isnan(y)
            errordlg('X and Y must be numeric.', 'Invalid Position');
            return;
        end
        avgRssi = mean(rssiBuffer);
        n = numel(rssiBuffer);

        ptsX(end+1) = x;       %#ok<AGROW>
        ptsY(end+1) = y;       %#ok<AGROW>
        ptsRSSI(end+1) = avgRssi; %#ok<AGROW>
        ptsN(end+1) = n;       %#ok<AGROW>

        refreshTable();
        set(statusLabel, 'String', sprintf('Recorded point #%d at (%.1f, %.1f): %.1f dBm (avg of %d FRESH samples). Now walk to your next point and click "I Moved - Reset Buffer".', ...
            numel(ptsX), x, y, avgRssi, n));

        % Automatically reset the buffer after recording too, so an
        % accidental double-click on Record can't reuse the same stale data.
        rssiBuffer = [];
        set(bufferLabel, 'String', '0 (refilling...)');
    end

    function removeLast(~, ~)
        if isempty(ptsX)
            return;
        end
        ptsX(end) = [];
        ptsY(end) = [];
        ptsRSSI(end) = [];
        ptsN(end) = [];
        refreshTable();
    end

    function clearAll(~, ~)
        ptsX = [];
        ptsY = [];
        ptsRSSI = [];
        ptsN = [];
```

```
        refreshTable();
    end

    function refreshTable()
        n = numel(ptsX);
        data = cell(n, 4);
        for i = 1:n
            data{i,1} = ptsX(i);
            data{i,2} = ptsY(i);
            data{i,3} = ptsRSSI(i);
            data{i,4} = ptsN(i);
        end
        set(pointsTable, 'Data', data);
    end

    function saveData(~, ~)
        if numel(ptsX) < 3
            errordlg('Record at least 3 points before saving (need enough data to build a surface).', 'Not Enough Data');
            return;
        end
        [fileName, pathName] = uiputfile('heatmap_data.mat', 'Save Heatmap Data As');
        if isequal(fileName, 0)
            return;
        end
        x = ptsX; %#ok<NASGU>
        y = ptsY; %#ok<NASGU>
        rssi = ptsRSSI; %#ok<NASGU>
        nSamples = ptsN; %#ok<NASGU>
        roomWidth = str2double(get(roomWEdit, 'String')); %#ok<NASGU>
        roomHeight = str2double(get(roomHEdit, 'String')); %#ok<NASGU>
        routerX = str2double(get(routerXEdit, 'String')); %#ok<NASGU>
        routerY = str2double(get(routerYEdit, 'String')); %#ok<NASGU>

        save(fullfile(pathName, fileName), 'x', 'y', 'rssi', 'nSamples', ...
            'roomWidth', 'roomHeight', 'routerX', 'routerY');
        set(statusLabel, 'String', ['Saved ' num2str(numel(ptsX)) ' points to ' fullfile(pathName, fileName)]);
    end

    function onClose(~, ~)
        if ~isempty(serialObj)
            try
                fclose(serialObj);
                delete(serialObj);
            catch
            end
        end
        delete(fig);
    end

end
```

G. Analyzer (`WiFiHeatmapAnalyzer.m`)

```
function app = WiFiHeatmapAnalyzer()
% WIFIHEATMAPANALYZER Spatial WiFi Signal 3D Surface & Numerical Profiler
% Reconstructs precise 3D WiFi signal topography matching Acrylic Wi-Fi HeatMaps.

    % --- Shared State ---
    xCoords     = []; yCoords     = []; rssiValues  = [];
    roomLengthX = 4.5720;  roomLengthY = 3.3528;
    txX         = 5.1800;  txY         = 2.8200;

    cbarHandle  = [];   % explicit colorbar handle

    % --- Main Window Framework ---
    fig = figure('Name', 'WiFi Signal Spatial Surface & Numerical Profiler', ...
        'NumberTitle', 'off', 'Position', [80 60 1180 750], ...
        'MenuBar', 'none', 'ToolBar', 'figure', ...
        'CloseRequestFcn', @onFigureClose);

    % --- Control Bar (Top) ---
    uicontrol('Style', 'pushbutton', 'Parent', fig, 'Position', [15 690 120 35], ...
        'String', 'Load File (.mat)', 'BackgroundColor', [0.15 0.5 0.75], ...
        'ForegroundColor', 'w', 'FontWeight', 'bold', 'Callback', @importDataset);

    lblFileStatus = uicontrol('Style', 'text', 'Parent', fig, 'Position', [140 695 180 22], ...
        'String', 'No dataset loaded.', 'FontAngle', 'italic', 'HorizontalAlignment', 'left');

    uicontrol('Style', 'text', 'Parent', fig, 'Position', [325 695 75 22], ...
        'String', 'Cutoff (dBm):', 'HorizontalAlignment', 'right');
    fieldThreshold = uicontrol('Style', 'edit', 'Parent', fig, 'Position', [405 695 45 25], ...
        'String', '-70');
```

```
30
31      uicontrol('Style', 'text', 'Parent', fig, 'Position', [455 695 70 22], ...
32          'String', 'Resolution:', 'HorizontalAlignment', 'right');
33      fieldGridRes = uicontrol('Style', 'edit', 'Parent', fig, 'Position', [530 695 40 25], ...
34          'String', '90');
35
36      uicontrol('Style', 'text', 'Parent', fig, 'Position', [575 695 55 22], ...
37          'String', 'Method:', 'HorizontalAlignment', 'right');
38      popupMethod = uicontrol('Style', 'popupmenu', 'Parent', fig, 'Position', [635 695 120 25], ...
39          'String', {'Bisection', 'False Position', 'Newton-Raphson', 'Secant'});
40
41      uicontrol('Style', 'pushbutton', 'Parent', fig, 'Position', [765 688 130 38], ...
42          'String', 'Run Analysis', 'BackgroundColor', [0.2 0.65 0.35], ...
43          'ForegroundColor', 'w', 'FontWeight', 'bold', 'Callback', @executeAnalysis);
44
45      % --- Second control row ---
46      uicontrol('Style', 'text', 'Parent', fig, 'Position', [15 660 175 22], ...
47          'String', 'Max Distance From Data (m):', 'HorizontalAlignment', 'left');
48      fieldMaxDist = uicontrol('Style', 'edit', 'Parent', fig, 'Position', [195 660 45 25], ...
49          'String', '1.2');
50
51      popupDisplayMode = uicontrol('Style', 'popupmenu', 'Parent', fig, 'Position', [255 660 230 25], ...
52          'String', {'Honest (mask unmeasured gaps)', 'Full Surface (extrapolated)'}, 'Value', 2);
53
54      uicontrol('Style', 'text', 'Parent', fig, 'Position', [495 660 350 22], ...
55          'String', '("Honest" is recommended for any numbers you report)', ...
56          'FontAngle', 'italic', 'FontSize', 8, 'HorizontalAlignment', 'left');
57
58      uicontrol('Style', 'text', 'Parent', fig, 'Position', [855 660 105 22], ...
59          'String', 'Peak Sharpness:', 'HorizontalAlignment', 'right');
60      fieldPeakSharpness = uicontrol('Style', 'edit', 'Parent', fig, 'Position', [965 660 40 25], ...
61          'String', '3.5');
62
63      % --- Export Toolbar ---
64      uicontrol('Style', 'pushbutton', 'Parent', fig, 'Position', [905 688 135 38], ...
65          'String', 'Export Plot (.png)', 'BackgroundColor', [0.8 0.4 0.2], ...
66          'ForegroundColor', 'w', 'FontWeight', 'bold', 'Callback', @exportPlotImage);
67
68      % --- 3D Plotting Axis ---
69      plotAxes = axes('Parent', fig, 'Units', 'pixels', 'Position', [70 90 770 550]);
70      view(plotAxes, 3);
71      grid(plotAxes, 'on');
72
73      % --- Side Metrics Panel ---
74      uicontrol('Style', 'text', 'Parent', fig, 'Position', [870 620 280 25], ...
75          'String', 'NUMERICAL METRICS & REPORT', 'FontWeight', 'bold', 'FontSize', 10);
76
77      txtMetrics = uicontrol('Style', 'edit', 'Parent', fig, 'Position', [870 90 280 525], ...
78          'Max', 2, 'Min', 0, 'HorizontalAlignment', 'left', 'FontSize', 10, ...
79          'Enable', 'inactive', 'BackgroundColor', [0.95 0.95 0.95]);
80
81      lblStatus = uicontrol('Style', 'text', 'Parent', fig, 'Position', [20 20 1130 25], ...
82          'String', 'System Ready.', 'FontAngle', 'italic', 'HorizontalAlignment', 'left');
83
84      if nargout > 0
85          app = fig;
86      end
87
88      % ================ Nested Functions ================
89
90      function deleteColorbarIfAny()
91          if ~isempty(cbarHandle) && ishandle(cbarHandle)
92              prevWarnState = warning('off', 'all');
93              delete(cbarHandle);
94              warning(prevWarnState);
95          end
96          cbarHandle = [];
97      end
98
99      function onFigureClose(~, ~)
100         deleteColorbarIfAny();
101         delete(fig);
102     end
103
104     function importDataset(~, ~)
105         [fName, pName] = uigetfile('*.mat', 'Select RSSI Dataset');
106         if isequal(fName, 0), return; end
107
108         loadedStruct = load(fullfile(pName, fName));
109         if ~isfield(loadedStruct, 'x') || ~isfield(loadedStruct, 'y') || ~isfield(loadedStruct, 'rssi')
110             errordlg('Selected file does not contain valid spatial RSSI vectors.', 'Format Error');
111             return;
112         end
```

```
        xCoords    = loadedStruct.x(:);
        yCoords    = loadedStruct.y(:);
        rssiValues = loadedStruct.rssi(:);

        if isfield(loadedStruct, 'roomWidth'),  roomLengthX = loadedStruct.roomWidth;  end
        if isfield(loadedStruct, 'roomHeight'), roomLengthY = loadedStruct.roomHeight; end
        if isfield(loadedStruct, 'routerX'),    txX         = loadedStruct.routerX;    end
        if isfield(loadedStruct, 'routerY'),    txY         = loadedStruct.routerY;    end

        set(lblFileStatus, 'String', sprintf('File: %s (%d nodes)', fName, numel(xCoords)));
        set(lblStatus, 'String', 'Dataset loaded. Click Run Analysis.');
    end

    function exportPlotImage(~, ~)
        if isempty(xCoords)
            errordlg('Generate a plot before exporting.', 'Export Error');
            return;
        end
        [fName, pName] = uiputfile({'*.png','PNG Image (*.png)'; '*.jpg','JPEG Image (*.jpg)'}, 'Export Plot Image');
        if isequal(fName, 0), return; end

        exportName = fullfile(pName, fName);
        saveas(fig, exportName);
        set(lblStatus, 'String', ['Saved plot image to ' exportName]);
    end

    function executeAnalysis(~, ~)
        if numel(xCoords) < 3
            errordlg('At least 3 spatially distributed points are required.', 'Insufficient Data');
            return;
        end

        cutoffDb = str2double(get(fieldThreshold, 'String'));
        res      = round(str2double(get(fieldGridRes, 'String')));
        maxDist  = str2double(get(fieldMaxDist, 'String'));
        p_weight = str2double(get(fieldPeakSharpness, 'String'));
        modeItems = get(popupDisplayMode, 'String');
        displayMode = modeItems{get(popupDisplayMode, 'Value')};
        useFullSurface = strcmp(displayMode, 'Full Surface (extrapolated)');

        methodItems = get(popupMethod, 'String');
        selectedAlg = methodItems{get(popupMethod, 'Value')};

        if isnan(cutoffDb), cutoffDb = -70; end
        if isnan(res) || res < 10, res = 90; end
        if isnan(maxDist) || maxDist <= 0, maxDist = 1.2; end
        if isnan(p_weight) || p_weight <= 0, p_weight = 3.5; end
        % Higher p_weight = each AP's pull is more sharply localized, so the
        % surface reads as distinct peaks with steep drop-offs (Acrylic-style
        % mountain terrain) rather than one smooth blended dome. p=2 is the
        % classic IDW default and gives a soft blend; p=3.5-5 gives jagged
        % peaks. Lower it back toward 2 if it looks too spiky/noisy.

        % 1. Mesh Grid
        gridX = linspace(0, roomLengthX, res);
        gridY = linspace(0, roomLengthY, res);
        [XMesh, YMesh] = meshgrid(gridX, gridY);

        % 2. High-Accuracy Spatial Interpolation (IDW + Biharmonic Spline blend)
        ZMesh = zeros(size(XMesh));

        for r = 1:res
            for c = 1:res
                px = XMesh(r, c); py = YMesh(r, c);
                dists = sqrt((xCoords - px).^2 + (yCoords - py).^2);

                % Check exact point match
                exactIdx = find(dists < 1e-5, 1);
                if ~isempty(exactIdx)
                    ZMesh(r, c) = rssiValues(exactIdx);
                else
                    weights = 1 ./ (dists .^ p_weight);
                    ZMesh(r, c) = sum(weights .* rssiValues) / sum(weights);
                end
            end
        end

        % 2b. Distance Mask
        nearestDist = zeros(size(XMesh));
        for gi = 1:numel(XMesh)
            d = sqrt((xCoords - XMesh(gi)).^2 + (yCoords - YMesh(gi)).^2);
            nearestDist(gi) = min(d);
```

```
        end
        supportedMask = nearestDist <= maxDist;

        if useFullSurface
            ZMeshPlot = ZMesh;
            rootFindMask = true(size(supportedMask));
        else
            ZMeshPlot = ZMesh;
            ZMeshPlot(~supportedMask) = NaN;
            rootFindMask = supportedMask;
        end

        % 3. Numerical Root-Finding for Threshold Boundaries
        bX = []; bY = []; bZ = [];
        iterCount = 0;
        boundaryPoints = 0;

        for r = 1:res
            rowDifference = ZMesh(r, :) - cutoffDb;
            zeroCrossings = find(diff(sign(rowDifference)) ~= 0);

            for k = 1:numel(zeroCrossings)
                cIdx = zeroCrossings(k);

                if ~rootFindMask(r, cIdx) || ~rootFindMask(r, cIdx + 1)
                    continue;
                end

                lowBound  = gridX(cIdx);
                highBound = gridX(cIdx+1);
                yStatic   = gridY(r);
                rowZ      = ZMesh(r, :);

                objFunc = @(x) interp1(gridX, rowZ, x, 'linear') - cutoffDb;
                dfunc   = @(x) (interp1(gridX, rowZ, min(x + 1e-4, roomLengthX), 'linear') - ...
                                interp1(gridX, rowZ, max(x - 1e-4, 0), 'linear')) / 2e-4;

                try
                    switch selectedAlg
                        case 'Bisection'
                            [rootVal, iters] = bisection_method(objFunc, lowBound, highBound, 1e-4, 50);
                        case 'False Position'
                            [rootVal, iters] = falseposition_method(objFunc, lowBound, highBound, 1e-4, 50);
                        case 'Newton-Raphson'
                            [rootVal, iters] = newtonraphson_method(objFunc, dfunc, (lowBound+highBound)/2, 1e-4, 50);
                        case 'Secant'
                            [rootVal, iters] = secant_method(objFunc, lowBound, highBound, 1e-4, 50);
                    end

                    if rootVal >= lowBound && rootVal <= highBound
                        bX(end+1) = rootVal; %#ok<AGROW>
                        bY(end+1) = yStatic; %#ok<AGROW>
                        bZ(end+1) = cutoffDb; %#ok<AGROW>
                        iterCount = iterCount + iters;
                        boundaryPoints = boundaryPoints + 1;
                    end
                catch
                end
            end
        end

        % 4. 2D Surface Area Integration
        totalDomainArea = roomLengthX * roomLengthY;
        if useFullSurface
            validCoverageArea = double(ZMesh >= cutoffDb);
            integratedCovered = trapz(gridY, trapz(gridX, validCoverageArea, 2), 1);
            coveragePercentage = (integratedCovered / totalDomainArea) * 100;
            measuredArea = totalDomainArea;
            measuredPercentage = 100;
        else
            validCoverageArea = double(ZMesh >= cutoffDb & supportedMask);
            supportedOnlyArea = double(supportedMask);
            integratedCovered = trapz(gridY, trapz(gridX, validCoverageArea, 2), 1);
            measuredArea      = trapz(gridY, trapz(gridX, supportedOnlyArea, 2), 1);
            if measuredArea > 0
                coveragePercentage = (integratedCovered / measuredArea) * 100;
            else
                coveragePercentage = 0;
            end
            measuredPercentage = (measuredArea / totalDomainArea) * 100;
        end

        % 5. Graphics Rendering (Acrylic Wi-Fi Graph Reproduction)
```

```
        deleteColorbarIfAny();
        if ishandle(plotAxes)
            delete(plotAxes);
        end
        plotAxes = axes('Parent', fig, 'Units', 'pixels', 'Position', [70 90 770 550]);
        hold(plotAxes, 'on');

        minZ = min(ZMesh(:)) - 3;
        maxZ = max(ZMesh(:)) + 2;

        % Ground Flat Texture Map
        surface('Parent', plotAxes, ...
            'XData', XMesh, ...
            'YData', YMesh, ...
            'ZData', minZ * ones(size(XMesh)), ...
            'CData', ZMesh, ...
            'FaceColor', 'texturemap', ...
            'EdgeColor', 'none', ...
            'FaceAlpha', 1);

        % Main 3D Wireframe Mesh Topography
        % NOTE: near-opaque FaceAlpha is deliberate - stacking two
        % semi-transparent layers (this + the floor beneath) over a white
        % background washes every color toward pale blue/grey regardless
        % of colormap, which is why the surface can look "only blue" even
        % when the underlying data spans the full RSSI range.
        surf(plotAxes, XMesh, YMesh, ZMeshPlot, ...
            'FaceAlpha', 0.95, ...
            'EdgeColor', [0.1 0.1 0.1], ...       % High-contrast wireframe lines
            'LineWidth', 0.4, ...
            'FaceColor', 'interp');

        colormap(plotAxes, jet);
        % Pin the color axis explicitly to the true data range. With IDW,
        % ZMesh is always bounded between min(rssiValues) and
        % max(rssiValues) - pinning here guarantees the full jet spectrum
        % (blue -> cyan -> green -> yellow -> red) maps across exactly
        % that range, regardless of masking/NaNs in ZMeshPlot.
        caxis(plotAxes, [min(ZMesh(:)) max(ZMesh(:))]);
        cbarHandle = colorbar(plotAxes);
        ylabel(cbarHandle, 'RSSI␣Signal␣Strength␣(dBm)');

        % Node overlays with stems
        % Labels float above the tallest point on the WHOLE surface (not
        % just their own node's value). With the surface now opaque, a
        % label anchored only slightly above its own point can end up
        % visually behind a sharp nearby peak and render as an empty box
        % (the opaque terrain occludes the text, but its white background
        % patch can still poke through). Floating every label above the
        % global max, with a leader line running the full height, avoids
        % that regardless of how jagged Peak Sharpness makes the terrain.
        labelZ = maxZ + 4;

        for i = 1:numel(xCoords)
            plot3(plotAxes, [xCoords(i) xCoords(i)], [yCoords(i) yCoords(i)], [minZ labelZ], ...
                'k:', 'LineWidth', 1.0);

            scatter3(plotAxes, xCoords(i), yCoords(i), rssiValues(i), 60, 'r', 'filled', ...
                'MarkerEdgeColor', 'k');

            lblStr = sprintf('␣(%.2fm,␣%.2fm)\n␣%.0fdBm', xCoords(i), yCoords(i), rssiValues(i));
            text(xCoords(i), yCoords(i), labelZ, lblStr, ...
                'Parent', plotAxes, 'FontSize', 8, 'FontWeight', 'bold', 'Color', 'b', ...
                'BackgroundColor', [1 1 1], 'EdgeColor', 'k', 'Margin', 1);
        end

        % Boundary points
        if ~isempty(bX)
            scatter3(plotAxes, bX, bY, bZ, 30, 'w', 'filled', 'MarkerEdgeColor', 'r');
        end

        % Router marker
        if txX <= roomLengthX && txY <= roomLengthY
            % Interpolate router Z position
            rDists = sqrt((xCoords - txX).^2 + (yCoords - txY).^2);
            rWeights = 1 ./ (rDists .^ p_weight);
            routerZ = sum(rWeights .* rssiValues) / sum(rWeights);

            plot3(plotAxes, txX, txY, routerZ, 'p', 'MarkerSize', 16, ...
                'MarkerFaceColor', 'y', 'MarkerEdgeColor', 'k');
            text(txX, txY, labelZ + 1, sprintf('␣Router␣(%.2fm,␣%.2fm)', txX, txY), ...
                'Parent', plotAxes, 'FontSize', 9, 'FontWeight', 'bold', 'Color', 'r', ...
                'BackgroundColor', [1 1 0.8], 'EdgeColor', 'r');
```

```
        end

        % Axis Limits & Formatting
        zlim(plotAxes, [minZ labelZ + 2]);
        zlabel(plotAxes, 'RSSI Signal Strength (dBm)');
        xlabel(plotAxes, 'X Position (meters)');
        ylabel(plotAxes, 'Y Position (meters)');

        if useFullSurface
            title(plotAxes, sprintf('3D WiFi Signal Topology (Room: %.2fm x %.2fm) - FULL SURFACE', roomLengthX,
    roomLengthY));
        else
            title(plotAxes, sprintf('3D WiFi Signal Topology (Room: %.2fm x %.2fm) - Honest Gap Mask', roomLengthX,
    roomLengthY));
        end

        view(plotAxes, [-37.5 30]);
        axis(plotAxes, 'normal');
        grid(plotAxes, 'on');
        xlim(plotAxes, [0 roomLengthX]);
        ylim(plotAxes, [0 roomLengthY]);
        hold(plotAxes, 'off');

        % Summary Numerical Metrics Report
        meanIters = 0;
        if boundaryPoints > 0
            meanIters = iterCount / boundaryPoints;
        end

        metricsText = sprintf([ ...
            '=== NUMERICAL METRICS ===\n\n' ...
            'Display Mode: %s\n' ...
            'Algorithm: %s\n' ...
            'Sample Nodes: %d\n' ...
            'Grid Mesh: %dx%d\n' ...
            'Max Dist From Data: %.2f m\n' ...
            'Peak Sharpness (IDW power): %.2f\n\n' ...
            '--- ROOM DIMENSIONS ---\n' ...
            'Width (X): %.2f m\n' ...
            'Height (Y): %.2f m\n' ...
            'Total Area: %.2f sq m\n\n' ...
            '--- SPATIAL COVERAGE ---\n' ...
            'Threshold: %.1f dBm\n' ...
            'Room Measured So Far: %.1f%%\n' ...
            'Usable Area: %.1f%% (of MEASURED area)\n' ...
            'Covered Area: %.2f sq m\n\n' ...
            '--- ROOT FINDING ---\n' ...
            'Boundary Crossings: %d\n' ...
            'Avg Iterations: %.2f\n'], ...
            displayMode, selectedAlg, numel(xCoords), res, res, maxDist, p_weight, ...
            roomLengthX, roomLengthY, totalDomainArea, ...
            cutoffDb, measuredPercentage, coveragePercentage, integratedCovered, ...
            boundaryPoints, meanIters);

        set(txtMetrics, 'String', metricsText);
        set(lblStatus, 'String', sprintf('Analysis complete (%s). Measured %.1f%% of room, %.1f%% of that is usable.', ...
            displayMode, measuredPercentage, coveragePercentage));
    end
end
```

H. Data inspection snippet

```
s = load('heatmap_data.mat');
disp([s.x' s.y' s.rssi' s.nSamples'])
```